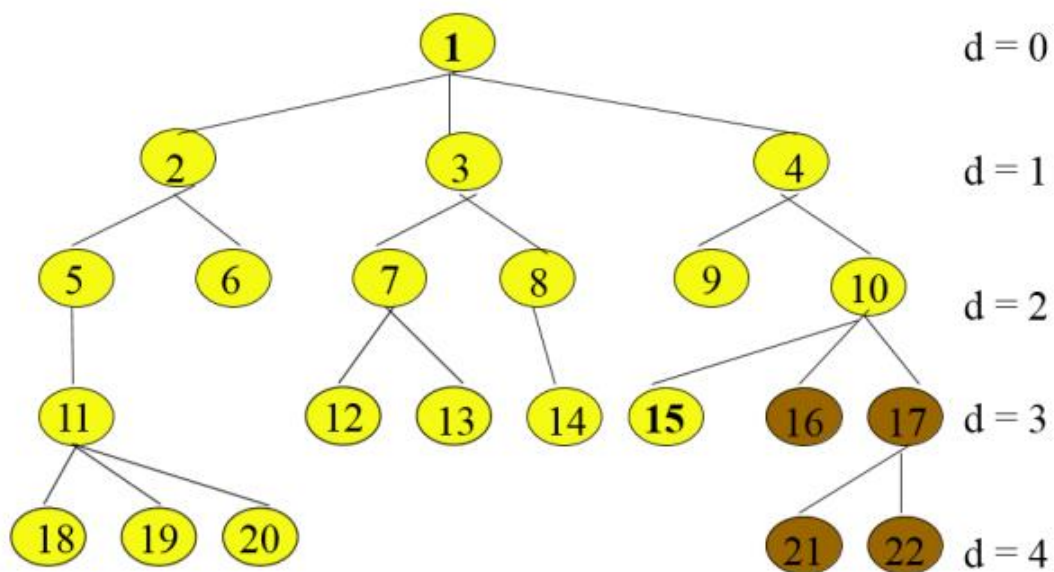


Lista 1 – Busca

Edryck Freitas Nascimento – a2727617

1. Simule busca em largura e em profundidade no cenário de problema modelado pela árvore abaixo para encontrar os nós 10 e 20. Compare tanto o número de nós visitados quanto expandidos, por fim diga qual o uso de tempo e memória de cada algoritmo na sua execução.



Busca em Largura:

10: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10. Tem 10 nós visitados e 9 nós que foram expandidos (1, 2, 3, 4, 5, 6, 7, 8 e 9).

20: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20. Tem 20 nós visitados e 19 nós expandidos.

Busca em Profundidade:

10: 1, 2, 5, 11, 18, 19, 20, 6, 3, 7, 12, 13, 8, 14, 4, 9, 10. Tem 17 nós visitados e 16 expandidos.

20: 1, 2, 5, 11, 18, 19, 20. Tem 7 nós visitados e 6 expandidos.

Para o nó 10, em largura foi preciso visitar 10 nós apenas, ao contrário da busca em profundidade que preciso de 17. Já para o nó 20, em profundidade foi mais eficiente, precisando de apenas 7 nós, ao contrário da busca em largura que precisou de 20 nós. O uso de tempo para a de profundidade é $O(b^m)$, péssimo quando m é muito maior que d e a de largura é $O(b^{d+1})$.

A de profundidade é $O(b^*m)$, espaço linear, já a de largura é $O(b^{d+1})$ já que ela mantém todos na memória.

2. Defina com suas próprias palavras os seguintes termos:

- a. **Estado**
Situação de um momento
- b. **Espaço de estados**
Conjunto de todos os estados possíveis
- c. **Árvore de busca**
Uma estrutura que representa um processo de busca, a raiz é o estado inicial e cada ramo da árvore vai representar uma ação
- d. **Nó de busca**
Elemento de uma árvore que representa um estado, seu pai seria a ação que levou a ele
- e. **Estado objetivo**
Estado que representa a solução de um problema
- f. **Ação**
Operação que um agente pode executar para executar um estado para transforma o estado atual em outro
- g. **Função-sucessor**
Função que define quais são os estados vizinhos de um estado
- h. **Fator de branching**
Número médio de filhos gerados para cada nó na árvore
- i. **Custo do caminho**
Soma dos custos das ações ao longo de um caminho do estado atual até um outro estado
- j. **Solução do problema**
Sequência de ações que leva o estado inicial a um objetivo

3. Descreva o estado inicial, a função de teste, a função-sucessor e a função de custo para cada um seguintes problemas. Modele como algoritmos de busca poderiam resolver cada um destes problemas exemplificando como seria um estado, como são gerados novos estados e quando o algoritmo poderia encontrar a solução, utilize representações gráficas e desenhos para esboçar a modelagem e execução do algoritmo.

- a. Você tem que colorir um mapa plano usando somente 4 cores, de tal forma que duas regiões adjacentes não tenham a mesma cor.
<https://github.com/Edryck/Fundamentos-de-Inteligencia-Artificial>
- b. Você tem três jarros, medindo 12 litros, 8 litros e 3 litros e uma fonte de água. Você pode encher ou esvaziar os jarros de um para o outro ou no chão. Você quer medir exatamente um litro em qualquer jarro.
<https://github.com/Edryck/Fundamentos-de-Inteligencia-Artificial>
- c. Um macaco de meio metro de altura está em uma jaula onde algumas bananas estão suspensas à três metros e meio do chão.

Ele quer pegar as bananas. A jaula contém dois caixotes de um metro e meio cada que podem ser movidos e sobrepostos.

<https://github.com/Edryck/Fundamentos-de-Inteligencia-Artificial>

4. O que difere uma estratégia de busca A de uma estratégia de busca B e como se avalia geralmente as estratégias de busca (critérios)?

Seria a ordem de expansão dos nós e os critérios usados para avaliar são:

- Completude: Sempre encontra uma solução (se existir)?
- Otimalidade: Sempre encontra a solução de custo mais baixo?
- Complexidade (tempo): Número de nodos explorados?
- Complexidade (espaço): Número de nodos armazenados?

5. Explique rapidamente cada uma das estratégias abaixo, destacando qual nó da fronteira é visitado primeiro, como a fronteira se comporta e o desempenho para cada uma delas. Em seguida simule cada um deles para o problema do mapa da Romênia (de Arad a Bucareste).

a. Busca em profundidade

Pega o primeiro nó não expandido, guarda o caminho atual e os irmãos não visitados, assim, crescendo a fronteira pouco a pouco. Ela é rápida e econômica em memória, mas não é completa em espaços infinitos e não é ótima.

Por ordem alfabética: Arad – Sibiu – Fagaras – Bucareste. (Total de 450Km)

b. Busca em extensão (ou largura)

A expansão é nível por nível, a fronteira cresce bastante por estar guardando todos os nós do nível. Ela é completa e ótima quando os custos são iguais, mas não é o caso do mapa.

Nível 1: Arad; Nível 2: Sibiu; Nível 3: Fagaras; Nível 4: Bucharest

c. Busca em profundidade iterativa

Busca em profundidade com limite de profundidade, definindo uma constante l como limite, quando chega neste limite, para e volta pra o outro nó.

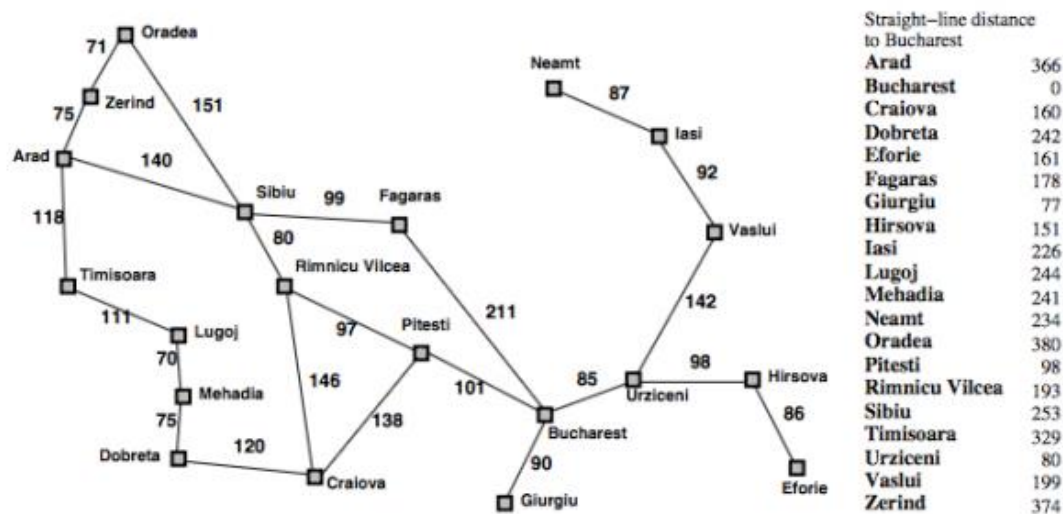
l = 0: Arad (falhou)

l = 1: Arad/Timisoara, Arad/Sibiu, Arad/Zerind (falhou)

l = 2: Arad/Timisoara/Lugoj, Arad/Sibiu/Rimnicu Vilcea, Arad/Sibiu/Fagaras, Arad/Zerind/Oradea (falhou)

l = 3: Arad/Timisoara/Lugoj/Mehadia, Arad/Sibiu/Rimnicu Vilcea/Craiova, Arad/Sibiu/Rimnicu Vilcea/Pitesti, Arad/Sibiu/Fagaras/Bucharest, Arad/Zerind/Oradea/Sibiu (Sucesso)

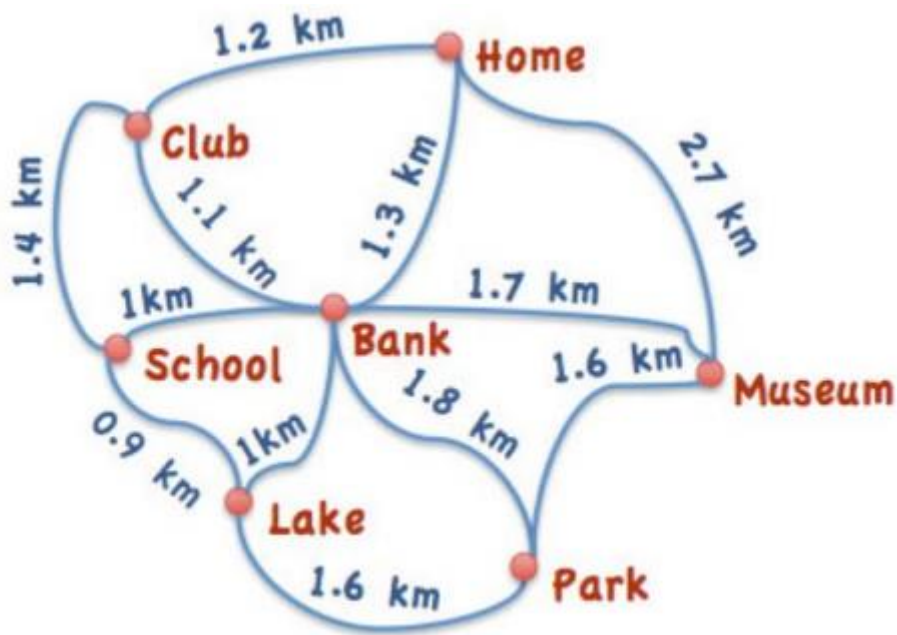
Romênia com custos por passo em km



6. Discorra sobre completude, otimalidade, custo de tempo e memória dos algoritmos Busca em Largura, Profundidade, Profundidade Limitada e Aprofundamento Iterativo. Existe alguma condição para que as estratégias sejam ótimas ou completas.

Estratégia	Completa?	Ótima?	Tempo	Memória
Largura	Sim (se b finito)	Sim, só se todos os custos de passo forem iguais	$O(b^d)$	$O(b^d)$
Profundidade	Não (espaços infinitos/loops), sim em espaços finitos com checagem de repetição	Não	$O(b^m)$	$O(bm)$
Profundidade limitada	Sim, só se o limite $\ell \geq$ profundidade da solução	Não	$O(b^\ell)$	$O(b\ell)$
Aprofundamento iterativo	Sim (se b finito)	Sim, se custos de passo iguais	$O(b^d)$	$O(bd)$

7. Considerando o mapa abaixo:



Utilize os algoritmos abaixo para sair de “School” e chegar em “Museum”. Qual dos algoritmos apresentou melhor resultado?

Busca em largura e busca em profundidade iterativa

a. Busca em Largura

Nível 1: School

Nível 2: Lake, Bank, Club

Nível 3: Park (via Lake), Home e Museum (Via Bank)

Pela menor distância: School, Bank, Museum

b. Profundidade

Se descer por ordem alfabética: School, Bank, Club, Home, Museum

c. Profundidade Iterativa (com limite de 4)

Limite 0, 1 e dois falham, no limite 3 fica School, Bank, Museum

8. Execute os algoritmos de busca em Largura e Profundidade em nível de código na IDE de sua preferência para o mapa da questão anterior. Colete os tempo de execução de cada algoritmo e diga qual foi o mais rápido.

<https://github.com/Edryck/Fundamentos-de-Inteligencia-Artificial>

Houve um empate, fiz o código em C e a diferença de tempo foi praticamente imperceptível, afinal, é um grafo bem pequeno.

Questao 8: BFS x DFS (School -> Museum) =====

Caminho: School -> Bank -> Museum

Nos visitados: 6 | expandidos: 5

Tempo de execucao: 0.0000000 s

Caminho: School -> Lake -> Park -> Museum

Nos visitados: 4 | expandidos: 3

Tempo de execucao: 0.0000000 s

9. Altere os algoritmos para criar o algoritmo de busca Profundidade limitada e Aprofundamento Iterativo e execute eles no mapa da Romênia (considera apenas as cidades que estão atrás de Bucharest) para sair de Arad e chegar em Bucarest. Teste os algoritmos com limites 2, 4 e 7.

<https://github.com/Edryck/Fundamentos-de-Inteligencia-Artificial>

```
==== Questao 9: Profundidade Limitada x Aprofundamento Iterativo ====
(Arad -> Bucareste, mapa reduzido ao lado oeste da Romenia)

--- Limite = 2 ---
[Prof. Limitada] Falhou (nao encontrou dentro do limite)
[Prof. Limitada] Expandidos: 4 | Tempo: 0.00000000 s
[Aprof. Iterativo] Falhou ate o limite maximo testado
[Aprof. Iterativo] Expandidos (somando todas as iteracoes): 5 | Tempo: 0.00000000 s

--- Limite = 4 ---
[Prof. Limitada] Caminho: Arad -> Sibiu -> Fagaras -> Bucareste
[Prof. Limitada] Expandidos: 8 | Tempo: 0.00000000 s
[Aprof. Iterativo] Caminho: Arad -> Sibiu -> Fagaras -> Bucareste
[Aprof. Iterativo] Encontrado no limite 3
[Aprof. Iterativo] Expandidos (somando todas as iteracoes): 11 | Tempo: 0.00000000 s

--- Limite = 7 ---
[Prof. Limitada] Caminho: Arad -> Timisoara -> Lugoj -> Mehadia -> Dobreta -> Craiova -> Pitesti -> Bucareste
[Prof. Limitada] Expandidos: 7 | Tempo: 0.00000000 s
[Aprof. Iterativo] Caminho: Arad -> Sibiu -> Fagaras -> Bucareste
[Aprof. Iterativo] Encontrado no limite 3
[Aprof. Iterativo] Expandidos (somando todas as iteracoes): 11 | Tempo: 0.00000000 s
```